W.02

The Poisson Distribution

Both Workshops W.00 *(*The Uniform Distribution*) and* W.01 *(*The Normal Distribution*) introduced distributions which are largely concerned with modelling* things. *We have discussed the use of distributions to model* probabilities*: especially in* W.01 *where we looked at use of the CDF for the normal distribution in some detail. The third distribution, and the topic of this workshop focuses exclusively on probability modelling: the Poisson distribution.*

Contents

Introduction . . . 2
- Aim . . . 2
- Learning Outcomes . . . 2

Defining the Poisson Distribution . . . 2
The Poisson Distribution in SciPy . . . 3
- Plotting the PMF and CDF . . . 3
- Extra Mile: Exploring PMFs and CDFs . . . 10

Quantiles: Using the Poisson Distribution . . . 10
- Determining Single Event Probabilities . . . 10
- Determining Multiple Event Probabilities . . . 11
- Determining Probability Ranges . . . 11
- Finding the Mean Rate . . . 12

Questions . . . 14
References . . . 15

Introduction

Aim

Both *Workshops W.00* (*The Uniform Distribution*) and *W.01* (*The Normal Distribution*) introduced distributions which are largely concerned with modelling *things*. We have discussed the use of distributions to model *probabilities*: especially in *W.01* where we looked at use of the CDF for the normal distribution in some detail. The third distribution, and the topic of this workshop focuses exclusively on probability modelling: the Poisson distribution.

The Poisson distribution also differs from the uniform and the normal distribution in that it *only* has a discrete form. That is the Poisson distribution is a *discrete probability distribution*. For many fields, simulation of processes can be built from a series of *events* linking *inputs* to *outputs* at discrete time intervals. This technique is called *discrete event simulation* (DES) (Forbus and Berleant, 2022), and underpins industrial tools such as FlexSim (Ubowska and Królikowski, 2024). Not all distributions used in DES follow a Poisson distribution[1], but the discrete nature of the distribution both fits the underlying nature of DES, and provides tools to easily model common scenarios. For example a Poisson distribution would be a valid model for the number of calls arriving at a call centre, assuming all calls are independent. In networking engineering, the Poisson distribution is also often using to model packet arrival times or other discrete events.

Learning Outcomes

- To understand the definition of the Poisson distribution, and how this definition can be used in NumPy and SciPy
- To quickly plot multiple datasets or distributions on charts in Matplotlib
- To apply your understanding to model simple probabilities using a Poisson distribution

Defining the Poisson Distribution

The Poisson distribution is a discrete probability distribution for *independent* events. The distribution expresses the probability of a given number of events occurring in a fixed interval of time if these events occur with a known constant mean rate[2]. It is defined by its constant mean rate, λ, which is also the value of the variance, at the quantile k. The probability mass function and cumulative distribution functions are given below

Mean: λ

Variance: λ

[1] And indeed the next *Workshop*, *W.03*, introduces the exponential distribution, which builds on the Poisson distribution studied here.

[2] *Note* 'time' could be another interval type (with a dimension greater than 1) such as number of events in a given area or volume. The main criteria is that events are independent of each other.

Probability Mass Function (PMF):

$$\begin{aligned} f(k;\lambda) &= P(X = k) \\ &= \frac{\lambda^k e^{-k}}{k!} \end{aligned}$$

for $k \in \{0, 1, 2, ...\}$ and $\lambda \in (0, \infty)$

Cumulative Distribution Function (CDF):

$$\begin{aligned} F(k;\lambda) &= e^{-\lambda} \sum_{n=0}^{\lfloor k \rfloor} \frac{\lambda^n}{n!} \\ &= \frac{\Gamma(\lfloor k+1 \rfloor, \lambda)}{k!} \\ &= \frac{1}{\Gamma(\lfloor k \rfloor + 1)} \int_{\lambda}^{\infty} t^{\lfloor k \rfloor} e^{-t} \end{aligned}$$

where $\Gamma(x, y)$ is the upper incomplete Gamma function, and $\lfloor k \rfloor$ is the floor function.

The Poisson distribution is a generalisation of the binomial distribution which is defined as

$$P(X = r) = {}^{n}C_r (1-p)^{n-r} p^r$$

where

- r is number of successes
- n is number of trials and
- p is probability of success

For large values of n with small values of p we can use the Poisson distribution with $\lambda = np$ as an approximation.

The Poisson Distribution in SciPy

Plotting the PMF and CDF

Using the Standard Method Calls In `scipy.stats` the `scipy.stats.poisson` class defines the Poisson distribution, using the same interface we have used previously.

Both the PMF and CDF are defined at the quantile k, with the mean `mu`, and the location `loc`. The core method prototypes are therefore `scipy.stats.poisson.pmf(k, mu, loc)`, or `scipy.stats.poisson.cdf(k, mu, loc)`.

By default for both the `pmf()` and `cdf()` methods the third parameter, `loc`, has the value `loc = 0`. The `loc` parameter effectively shifts the standardised Poisson from the mean at k for both the PMF and CDF. Both methods can therefore also be called as

- `scipy.stats.poisson.pmf(k - loc, mu)`, or
- `scipy.stats.poisson.cdf(k - loc, mu)`

For our plots and examples, we will set $\lambda = 2$, and leave the `loc` at the default (`0`), for the subsequent calls[3]

```
p_lambda = 2
```

Using the `stats()` method, and with the standard arguments for mean and variance, we then obtain the following values. Unlike previous distributions, however, for the Poisson distribution only have to set the mean, `p_lambda`, to obtain the correct distribution

```
mean, var = sp.stats.poisson.stats(p_lambda, moments="mv")

print(f"\nMean: {mean:.2f}\n" f"Variance: {var:.2f}\n")
```

```
Mean: 2.00
Variance: 2.00
```

Note the variance and the mean are the same: as we saw from the earlier definition.

We will now plot the PMF and CDF for different quantiles, which follows essentially the same process as we have used before. Therefore we will only briefly cover the necessary Python code, and concentrate on the output from the PMF and CDF distributions.

First we will define the range of quantiles to plot, from 0 to 10 in integer steps

```
quantiles = np.arange(0, 11, 1)
```

The PMF can then be plotted in Figure 1 for each quantile in `quantiles`, using the mean `p_lambda`, as follows. The only difference between previous plots in *Workshops W.00* (*The Uniform Distribution*) and *W.01* (*The Normal Distribution*) is the call to `scatter()` (instead of `plot()`) in `Line 28`. This creates a *scatter plot*, where each quantile, x, and PMF value, y is plotted separately. As with other PMF values, the output of the Poisson PMF is *discrete* and so the values are *not* connected. We can, though, still see the shape of the PMF in Figure 1.

```
# A Scatter Plot of the Poisson PMF where $\lambda = 2$
# For the Quantiles $k = 1, 2, \dots,20$
# fig:W02_PoissionPMF1

# Plot the result
plot.figure(figsize=(8, 4))

# Format the plot
plot.grid(True, which="both")
plot.xticks(quantiles)

# Label the axes
plot.xlabel("Quantile ($k$)")
```

[3] In Python 3, '`lambda`' is now a reserved word for anonymous functions. So the variable used here (and in later calls) will be called '`p_lambda`' for the mean, λ.

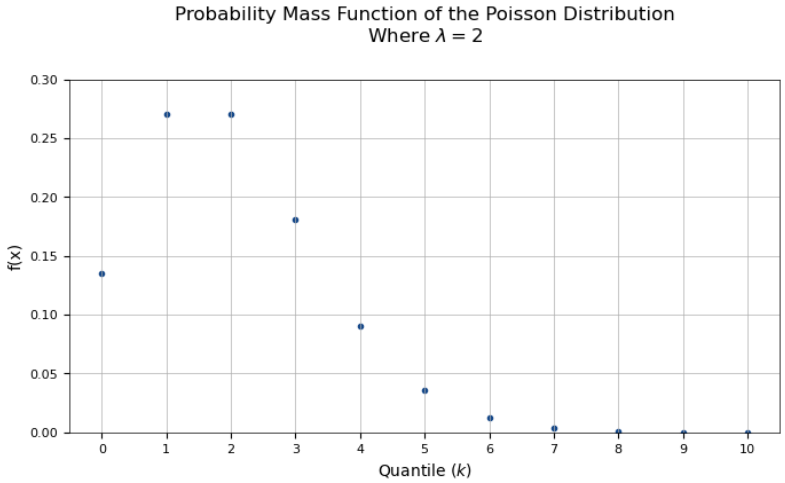


Figure 1: A Scatter Plot of the Poisson PMF where $\lambda = 2$ For the Quantiles $k = 1, 2, \ldots, 20$

```
14    plot.ylabel("f(x)")
15
16    # Label the plot
17    plot.title(
18        "Probability Mass Function of the Poisson Distribution\n"
19        "Where $\\lambda = $" + str(p_lambda),
20        pad=24.0,
21    )
22
23    # Set the axis limits
24    plot.ylim(0, 0.3)
25
26    # Display the graph
27    plot.scatter(
28        quantiles, sp.stats.poisson.pmf(quantiles, p_lambda)
29    )
```

Plotting the CDF in Figure 2 is essentially identical, apart from the method call in `Line 28`. Again we are also using a scatter plot for Figure 2, to show the values of the CDF are distinct.

```
1     # The Plot of the Poisson CDF where $\lambda = 20$
2     # For the Quantiles $k = 1, 2, \dots,20$
3     # fig:W02_PoissionCDF1
4
5     # Plot the result
6     plot.figure(figsize=(8, 4))
7
8     # Format the plot
9     plot.grid(True, which="both")
10    plot.xticks(quantiles)
```

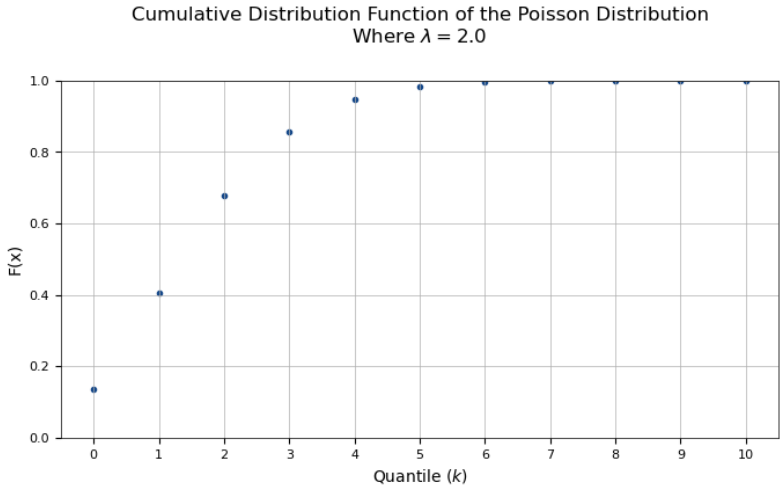


Figure 2: The Plot of the Poisson CDF where $\lambda = 20$ For the Quantiles $k = 1, 2, \ldots, 20$

```
11
12      # Label the axes
13      plot.xlabel("Quantile ($k$)")
14      plot.ylabel("F(x)")
15
16      # Label the plot
17      plot.title(
18          "Cumulative Distribution Function of the Poisson Distribution\n"
19          "Where $\\lambda = $" + str(mean),
20          pad=24.0,
21      )
22
23      # Set the axis limits
24      plot.ylim(0, 1.0)
25
26      # Display the graph
27      plot.scatter(
28          quantiles, sp.stats.poisson.cdf(quantiles, p_lambda)
29      )
```

Plotting Multiple Poisson Distributions To compare the distributions of different Poisson curves at different means, we will use an adaptation of techniques you have seen in *Workshops W.00* (*The Uniform Distribution*) and *W.01* (*The Normal Distribution*). In *Workshop W.00* we used successive calls to `plot.plot()` when adding multiple curves to a single figure. Whereas in *Workshop W.01* we introduced the method call to `plot.subplots()` to break up the display of quantiles into four separate sub-figures.

For the comparison of mean values, we will use 'sub-figures': but this time combine all the curves into a single figure. This technique is useful when you don't know quite how many curves you want to combine on a single plot, as we can add each curve programmatically.

We will start with an array of mean values we want to plot curves for

```
# generating PMF for different means
means = [0.5, 1, 2.25, 4]
```

We will now use `plot.subplots` as we did for *Workshop W.01*, but this time we will set the rows and columns to '1' in `Line 5`. This will force the generation of a single figure, instead of the multiple sub-figures we had last time. We will then walk the `means` array, creating a new plot via a call to `subfig.plot()` in `Line 8` to `Line 13`. We are also re-using the `quantiles` array we set-up before, and similarly using the call to `poisson.pmf()` to generate the curve at the specified `mean` value.

The rest of the code is much the same as last time. The main different is the use of a *format string*, '`o:`', in the calls to `subfig.plot()`. Format strings allow you to compactly specify the marker type, colour and the connecting line format as a single string. In this case it represents a circle marker ('o'), connected by a dotted line-style (':')[4]. We have left Matplotlib to pick the colour from the current colour cycler, so that we automatically get a different colour for each call. The final result should be in Figure 3.

```
# Plot of the PMF Values of the Quantiles for Four Poisson
# Distributions
# fig:W02_PoissionPMF2

figure, subfig = plot.subplots(ncols=1, nrows=1)

for index in range(4):
    subfig.plot(
        quantiles,
        sp.stats.poisson.pmf(quantiles, means[index]),
        "o:",
        label=f"$\\lambda$={means[index]:.2f}",
    )

# Format the plot
subfig.grid(True, which="both")
subfig.legend(loc="best")

subfig.set_xticks(quantiles)

# Set the axis limits
subfig.set_ylim(0, 0.7)

# Label the plot
figure.suptitle(
    "Probability Mass Function of Four Poisson Distributions\n",
)

# Label the axes
```

[4] The Matplotlib cheatsheets also provide a useful summary of available options, outside the official documentation.

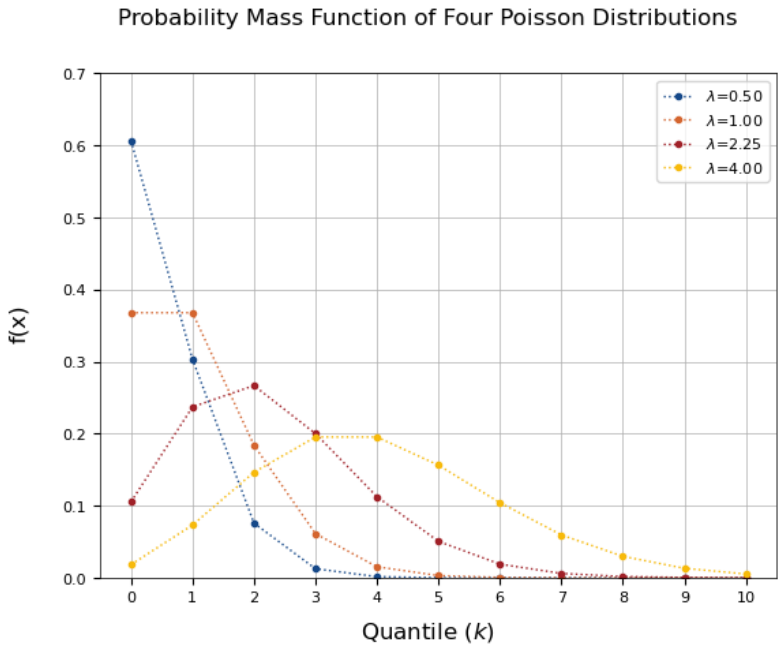


Figure 3: Plot of the PMF Values of the Quantiles for Four Poisson Distributions

```
    figure.supxlabel("Quantile ($k$)")
    figure.supylabel("f(x)")
```

Exactly the same techniques can then be used in Figure 4 to draw the equivalent CDF 'curve' for each of the mean values. Note that in both Figure 3 and Figure 4 the lines are to help with visualisation — they do *not* mean a continuous curve. The discrete values indicated by the circles are the actual values

Compared to the PMF of the uniform and normal distributions, the PMF for the Poisson distribution in Figure 3 shows distinct variations for different values of λ. For integer values of the mean ($\lambda = 1$ and $\lambda = 4$), we get two maximum values of the PMF at λ. You can also see this behaviour in Figure 4. Whereas the non integer values of λ result in a peak at one value.

```
# Plot of the CDF Values of the Quantiles for Four Poisson
# Distributions
# fig:W02_PoissionCDF2

figure, subfig = plot.subplots(1, 1)

for index in range(4):
    subfig.plot(
        quantiles,
        sp.stats.poisson.cdf(quantiles, means[index]),
        "o:",
```

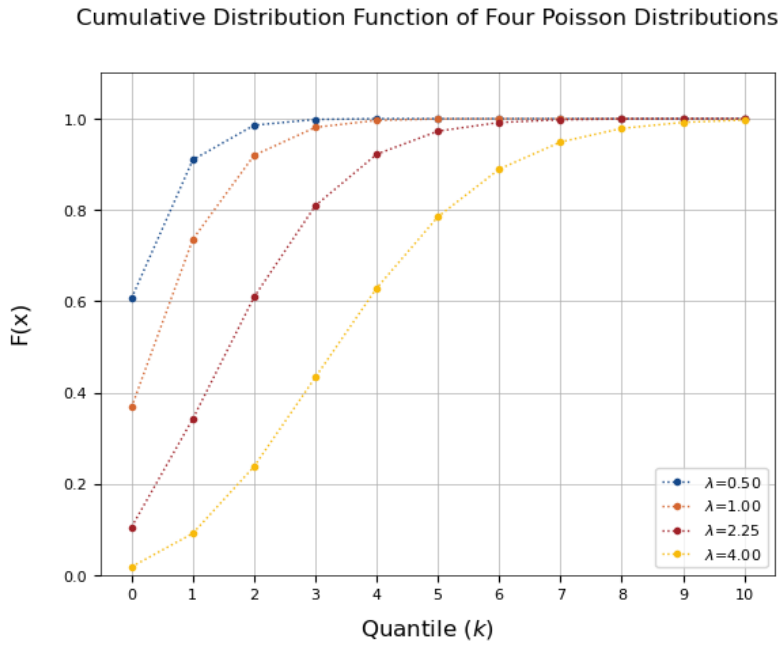


Figure 4: Plot of the CDF Values of the Quantiles for Four Poisson Distributions

```
            label=f"$\\lambda$={means[index]:.2f}",
        )

    # Format the plot
    subfig.grid(True, which="both")
    subfig.legend(loc="best")

    subfig.set_xticks(quantiles)

    # Set the axis limits
    subfig.set_ylim(0, 1.1)

    # Label the plot
    figure.suptitle(
        "Cumulative Distribution Function of Four Poisson "
        "Distributions\n",
    )

    # Label the axes
    figure.supxlabel("Quantile ($k$)")
    figure.supylabel("F(x)")
```

Extra Mile: Exploring PMFs and CDFs

You can explore more of the shape of both the PMF and CDF curves for the Poisson distribution by creating an interactive version of Figure 1 and Figure 2. Use the code we looked at in *W.01* (*The Normal Distribution*), and adapt it to use the Poisson distribution instead. This should help you explore the behaviour of the Poisson distribution at fractional values of λ more easily.

Quantiles: Using the Poisson Distribution

Determining Single Event Probabilities

The main use for the Poisson distribution is to estimate probabilities where a mean (constant) rate, λ, exists for a specific event. This use stems from the earlier definition of the Poisson distribution.

For example, imagine you are trying to work out whether to build a flood defence system. You would want to know whether the cost of building the defences outweighs the potential loss in allowing the river to flood. If the river only rarely floods, the cost of the flood defence is unlikely to be viable. So for our example we will say the river floods twice every 100 years. This gives a mean constant rate of flooding of $\lambda = 2$ for that 100 year period.

```
# Mean rate of flooding in a 100 year period
p_lambda = 2
```

Using the PMF of a Poisson distribution allows us to estimate probabilities at a *specific* quantile (k), for that given mean rate λ. For instance if we want to know the probability the river *won't* flood in the 100 year period we would want to evaluate the PMF at $P(k = 0)$

```
print(f"Probability of no floods:\
    {sp.stats.poisson.pmf(0, p_lambda): .3f}\n")
```

```
Probability of no floods:     0.135
```

Similarly to work out the probability of exactly three floods, $P(k = 3)$, we would use

```
print(f"Probability of three floods:\
    {sp.stats.poisson.pmf(3, p_lambda): .3f}\n")
```

```
Probability of three floods:     0.180
```

You should be able to estimate these values from Figure 2. And also from Figure 2 we know the probabilities of one, $P(k = 1)$, or two flooding events, $P(k = 2)$, will lie above these probabilities

```
print(f"Probability of one flood: \
    {sp.stats.poisson.pmf(1, p_lambda): .3f}")
print(f"Probability of two floods:\
    {sp.stats.poisson.pmf(2, p_lambda): .3f}\n")
```

```
Probability of one flood:      0.271
Probability of two floods:     0.271
```

Determining Multiple Event Probabilities

Just as we use the PMF for single event probabilities, the CDF of the Poisson distribution can be used for *multiple* event probabilities. Staying with the same example (so $\lambda = 2$), we can use the CDF in Figure 2 as follows.

Estimating probabilities *below* a quantile value k means summing from 0 to k. This is straightforward application of the CDF. For example to determine the probability of fewer than three flooding events we need to evaluate $P(k \leq 3)$

```
print(f"Probability of 3 floods or fewer:\
    {sp.stats.poisson.cdf(3, p_lambda): .3f}\n")
```

```
Probability of 3 floods or fewer:     0.857
```

Estimating probabilities *above* a specific quantile value requires finding the probability that is *not* below the required quantile. The easiest way to do that is to subtract the probability estimation on the CDF from 1.

For example to estimate the probability of more than four floods, $P(k > 4)$, we would use

```
print(f"Probability of more than 4 floods:\
    {1-sp.stats.poisson.cdf(4, p_lambda): .3f}\n")
```

```
Probability of more than 4 floods:     0.053
```

Determining Probability Ranges

Finding the range of probabilities for specific events may require using both the PMF and CDF, depending on the range of interest. For example if we are interested in knowing the probabilities of *exactly* 1 or 2 floods, $P(k \in \{1, 2\})$, we need both $P(k = 1)$ and $P(k = 2)$. The overall probability is then

$$P(k \in \{1, 2\}) = P(k = 1) + P(k = 2)$$

In Python, this looks like

```
print(f"Probability of 1 or 2 floods:\
    {sp.stats.poisson.pmf(1, p_lambda) +
    sp.stats.poisson.pmf(2, p_lambda): .3f}")
```

```
Probability of 1 or 2 floods:     0.541
```

Creative use of the python `range()` method can also help for more complex calculations. For example to estimate $P(k \in \{1, 2, 3\})$ we could use[5]

```
p_total = 0
for p in range(1, 4):
    p_total += sp.stats.poisson.pmf(p, p_lambda)

print(f"Probability of 1 to 3 floods: {p_total: .3f}")
```

```
Probability of 1 to 3 floods:  0.722
```

To estimate those probabilities we use quantiles in the same way we did for the normal distribution in *W.01* (*The Normal Distribution*). We can either estimate the probability at a specific value, using the

or we can estimate

Given on average a river overflow floods twice every 100 years, calculate, to 4 d.p., the probability that:

- There are no floods in 100 years $P(k = 0)$
- There are 3 or less floods in 100 years $P(k \leq 3)$
- There are more than 4 floods in 100 years $P(k > 4)$
- THere are 1 or 2 floods in 100 years $P(k \in \{1, 2\})$

Finding the Mean Rate

In the previous example, determining the constant mean rate is relatively straightforward. However in some cases we have to do a bit of work to determine what λ should be. For example in the following example we have two probabilities to consider

> On average, the probability of a integrated circuit chip being defective is 0.001. Given a batch of 800 chips what is the probability that:
>
> - none are defective, $P_1(k = 0)$
> - less than 2 are defective, $P_2(k < 2)$
> - at least 4 are defective, $P_3(k \geq 4)$
> - 1, 2 or 3 chips are defective, $P_4(k \in \{1, 2, 3\})$

[5] Note that `range()` computes the interval [[start, stop)), so we are using a `stop` value of 4, not 3 here.

However, the important assumption is that the *error probability*, P_{error}, is constant. So too is the batch size at 800. This means the error probability within each batch is also constant

$$\begin{aligned} P_{\text{batch error}} &= P_{\text{error}} \times 800 \\ &= 0.001 * 800 \\ &= 0.8 \end{aligned}$$

As long as the overall rate is constant, the Poisson distribution will work. In the example above, if P_{error} changes, or the batch size changes we will need a *different* Poisson distribution. And if the error rate or batch sizes is constantly changing, the Poisson distribution may not be applicable at all.

Assuming, though, the assumptions do apply, we can make the probability estimates as follows

```
# mean
p_lambda = 800 * 0.001

# using PMF to calculate fixed value
p1 = sp.stats.poisson.pmf(0, p_lambda)

# Using CDF to calculate less than or at least
p2 = sp.stats.poisson.cdf(1, p_lambda)
p3 = 1 - sp.stats.poisson.cdf(3, p_lambda)

# Use PMFs and sum for set of values
p4 = 0
for p in range(1, 4):
    p4 += sp.stats.poisson.pmf(p, p_lambda)

print(
    f"Probability of no defective chips:\t\t{p1: .3f}\n"
    f"Probability of less than 2 defective chips:\t{p2: .3f}\n"
    f"Probability of at least 4 defective chips:\t{p3: .3f}\n"
    f"Probability of 1, 2 or 3 defective chips:\t{p4: .3f}\n"
)
```

```
Probability of no defective chips:      0.449
Probability of less than 2 defective chips:  0.809
Probability of at least 4 defective chips:   0.009
Probability of 1, 2 or 3 defective chips:    0.542
```

Questions

Use Python to answer the following questions. Where appropriate give answers to 4 decimal places.

1. On average 4 vehicles arrive at a junction with traffic lights during the cycle of the lights. What is the probability:
   a. Exactly 5 vehicles arrive in a cycle $P(k = 5)$
   b. Less than 4 vehicles arrive in a cycle $P(k < 5)$)
   c. If the cycle allows only 5 vehicles through, what is the probability that all the waiting vehicles are not cleared in a cycle $P(k > 5)$

Answer:

```
# code for question 1(a) -- 1(c)
```

2. On average 3 calls per minute arrive at a call centre. What is the probability:
   a. 1 call arrives in a minute?
   b. Less than 3 calls arrive in a minute?
   c. 4 or more calls arrive in a minute?
   d. The number of calls that arrive is 2, 3 or 4?

Answer:

```
# code for question 2(a) -- 2(d)
```

3. Results show that 1 in 1000 transistors are defective on average. What is the probability that:
   a. There are no defective transistors in a batch of 750?
   b. There are 4 defective transistors in a batch of 2000?
   c. There are less than 5 defective transistors in a batch of 2500?
   d. There are 3 or more defective transistors in a batch of 1000?
   e. There is 1 or 2 defective transistors in a batch of 2000?

```
# code for question 3(a) -- 3(e)
```

4. On average a machine can process 4 packages in a minute. What is the probability that:
   a. In 10 minutes the process processes 45 packages?
   b. In 5 minutes 18 or fewer packages are processed?
   c. In 15 minutes between 55 and 65 packages are processed?
   d. In an hour at least 250 packages are processed?
   e. In an hour the number of packages processed is between 235 and 245?

Answer:

```
# code for question 4(a) -- 4(e)
```

References

$$

Forbus, J. J. and Berleant, D. (2022) Discrete-Event Simulation in Healthcare Settings: A Review. *Modelling*, 3 (4) December, pp. 417–433.

Ubowska, A. and Królikowski, T. (2024) New Technologies in the Educational Process on the Example of Flexsim® Software – the Digital Twin Solution. *Procedia Computer Science*, 246, pp. 4751–4758.